Bookings & Integrations Audit Report

Static Analysis and Logical Review of the Bookings & Partner Sync Modules

Date Scanned: 2026-06-15

FILES SCANNED

10

Core api routes, components & libs

LOGIC & SECURITY RISKS

3

Critical issues with cross-tenant data / RLS

CODE SMELLS & HOOKS

7

State mutations in effects, memoization failure

DEAD CODE INSTANCES

15

Unused imports, variables & dead state

⚠️ Critical Logic & Security Risks

The following logical issues are severe, potentially leading to cross-tenant data leaks, incorrect booking associations, or database corruption during synchronization.

1. Undefined Email Fallback in Partner Booking Sync

LOGICAL BUG

When looking up an existing guest in the target tenant organization, if the original guest email is null or empty, `originalGuest.email || undefined`

resolves to `undefined`. In Prisma Client, passing an `undefined` key skips the filter parameter completely. Consequently, the query retrieves **the first guest record** found in the target organization's database, linking the newly synced booking to a completely unrelated guest!

File: src/lib/bookingSync.ts:L87-94

```
// src/lib/bookingSync.ts
const existingGuest = await prisma.contact.findFirst({
  where: {
    organizationId: targetOrgId,
    contactType: 'guest',
    email: originalGuest.email || undefined, // ⟵ resolves to
    deletedAt: null
  }
});
```

2. Inbound Webhook Cross-Tenant Context Contamination

SECURITY RISK

In the Uplisting webhook endpoint, the fallback criteria in the organization query includes `{ isActive: true }`. Since this check is evaluated inside a database-wide `OR` statement, if a webhook payload lacks both `webhookOrgId` and `data.organization_slug`, the server will retrieve and run database mutations inside the context of the **first active organization returned by the query**, creating a severe cross-tenant data contamination risk.

File: src/app/api/webhooks/uplisting/route.ts:L53-61

```
// src/app/api/webhooks/uplisting/route.ts
const organization = await prisma.organization.findFirst({
  where: {
    OR: [
      { id: webhookOrgId || undefined },
      { slug: data.organization_slug || undefined },
      { isActive: true } // ⟵ Always matches first active ten
    ]
  }
});
```

3. Tenant Context Hard Block in Public Booking Engine

RUNTIME FRAGILITY

The public booking engine POST handler operates by finding the first active organization member to act as the context initiator for Prisma's Row-Level Security (RLS) context. If a tenant company suspends or removes all active user accounts, the booking engine will immediately throw a `500 Internal Server Error` ("Tenant organization has no active members"), preventing public guest bookings even if the property is open.

File: `src/app/api/bookings/engine/route.ts:L107-114`

🗑️ Dead Code & Unused Elements

The following imports, variables, and code statements are defined but never used, cluttering the modules and increasing maintainability overhead.

`src/app/api/bookings/route.ts` — UNUSED IMPORT

Unused Import: `prisma` is imported on line 3 but never used (queries execute within the tenant context transaction block `tx`).

`src/app/api/bookings/route.ts` — UNUSED PARAMETER

Unused Parameter: `request` in the `GET` request handler is defined but never read.

`src/components/bookings/BookingsList.tsx` — UNUSED IMPORT

Unused Icon: `Search` is imported from `lucide-react` but never rendered in the component.

src/components/bookings/BookingsList.tsx UNUSED STATE

Unused State Variable: `loading` state defined on line 389 is assigned but never read or referenced.

src/components/bookings/BookingsList.tsx DEAD VARIABLES

Unused Local Variables: `feeTotal` (L1237), `processingFee` (L1238), `merchantFee` (L1239), and `filteredBookings` (L1782) are created/assigned but never utilized.

src/components/bookings/MasterCalendar.tsx UNUSED IMPORT

Unused Icon: `Building` is imported from `lucide-react` but never rendered.

src/components/bookings/MasterCalendar.tsx DEAD HOOK

Unused Setter: `setDaysCount` is defined via `useState(12)` but never used, keeping the calendar views locked to exactly 12 columns.

src/components/channels/ChannelManagement.tsx UNUSED IMPORT

Unused Icon: `CheckCircle` is imported from `lucide-react` but never rendered in the markup.

src/components/channels/ChannelManagement.tsx REDUNDANT DESTRUCTURING

Redundant Destructuring: In `handleApplyMappings`, columns such as `commCol`, `basesCol`, `feeCol`, `vatCol`, `procCol`, etc., are destructured from `columnMappings` but never used inside the function (the entire object is passed to a helper instead).

`src/lib/bookingSync.ts` UNUSED PARAMETER

Unused Parameter: In `syncDeletedBooking`, `targetOrgId` is declared in the function signature but never referenced inside the method logic.

React & Performance Warnings

The React compiler and lint checks identified several hook violations, missing dependencies, and potential memory leaks in page-level components.

LINE	FILE	ISSUE TYPE	DESCRIPTION / RATIONALE
L1054	`BookingsList.tsx`	Cascading Render	Synchronous call of `setCurrency` directly within a `useEffect` body during initial mount can trigger multiple layout/rendering iterations, reducing browser responsiveness.
L464	`BookingsList.tsx`	Compiler Skip	React Compiler was unable to optimize `runAnalysis` due to failing to preserve the manual `useCallback` memoization boundaries.
L1758	`BookingsList.tsx`	Missing Dependency	`useCallback` hook relies on `getAutoScheduledTasks` but does not include it in its dependency array.

LINE	FILE	ISSUE TYPE	DESCRIPTION / RATIONALE
L117	`Reservations.tsx`	Cascading Render	Synchronous call to `setPropertySearchQuery` within an effect triggers immediately on render, causing double render cycles.
L152, 164	`Reservations.tsx`	Missing Dependency	`useMemo` hook relies on translation function `t` but lacks it in the dependency lists, which could cause stale UI headings when switching interface language.
L355	`ChannelManagement.tsx`	Missing Dependency	`useMemo` hook has a missing dependency: `getChannelNameOfRule`.

⚙️ Database Performance Bottlenecks

The following locations contain database calls that will degrade performance as the number of linked listings scale.

Unindexed Table-wide Scan for Reverse Mapping Lookup

PERFORMANCE

To perform a reverse sync lookup, `syncBookingToLinkedProperty` queries **every active property in the database** (`prisma.property.findMany({ where: { deletedAt: null } })`) and performs a manual array-find search in application memory. As the platform scales to hundreds or thousands of properties across all tenants, this query will load huge quantities of duplicate data and block CPU execution.

File: src/lib/bookingSync.ts:L63-69

Sequential Loop Sync with Sleep Timers in Cron

SUBOPTIMAL QUEUE

The cron sync handler processes all active tenant organizations sequentially in a single process loop with a hardcoded `delay(500)` in between. While safe for small portfolios, this synchronous execution scales linearly and will trigger function timeouts or gateway errors on serverless execution environments (like Vercel).

File: src/app/api/cron/sync-bookings/route.ts:L51-114

Report generated by Code Auditor Subagent. All scan information compiled from static typescript compiler settings and ESLint AST rules.